Where Does the Time Go? A Pre-Registered Study of Per-Step Cost in LLM Inference Serving

Patrick Niyongabo
Independent researcher, Portland, Oregon
https://github.com/pniyongabo/BottleneckShift

October 2026

Abstract

When an LLM inference server slows down under load, it is often unclear where the time goes. We measured vLLM serving a 0.5B-parameter model on one GPU in a series of pre-registered experiments totaling 18,900 measured requests. Most runs were observed from the client, the server's phase metrics, and GPU and CPU telemetry, and the last experiments added a per-step scheduler log. Every prediction was published before its data and scored after.

First, from 8 to 32 concurrent requests, past throughput saturation, the added latency moves into each generation step: the share of latency before the first token falls from 0.44 to 0.24 while median inter-token latency rises from 4.9 to 55 ms. Second, the scheduler's token budget decides where that delay appears. At 32 concurrent requests, a larger budget gives a 7 ms median and 18% more throughput but a 169 ms 99th percentile, and with the smallest budget queueing takes two thirds of each request's latency. Third, with one request at a time, a fixed-size prompt chunk costs up to 1.6× more at deeper context, in line with a count of attention FLOPs. A pre-registered test that a step's cost tracks its scheduled tokens was rejected at 8 concurrent requests, and its pass at 32 used a timing measure that we later found is confounded with the previous step.

Of 59 scored predictions, 40 held. We report the failures in full, including that measurement flaw. All GPU time for the study so far cost $2.04.

1 Introduction

LLM serving systems batch many requests through one model. Each engine step generates one token for every running request and may also process part of a new request's prompt [4, 9, 17]. Service targets are usually written in terms of time to first token (TTFT) and time per output token [19]. A user feels both the wait for the first token and how smoothly the rest stream. An operator who sees latency rise therefore needs to know which phase grew and why. Past saturation the excess work is the same either way. The scheduler decides whether it appears as waiting before the first token or as slower steps, and its settings move it between the two.

This report asks under which conditions the dominant source of client-visible delay changes, and which measurements can tell the causes apart. We answer it for one setup, vLLM serving Qwen2.5-0.5B-Instruct on one RTX A4000. We use a method designed to make negative results visible. Each experiment's predictions and thresholds were published before its data were collected, runs were measured from three independent views, and every prediction was scored afterward, including the ones that failed.

Our contributions are:

- *A pre-registered, three-view measurement method* for LLM serving, with an opt-in per-step scheduler log that records how many tokens each engine step processed and when its results returned (Section 3).
- *Where latency goes under load:* from C8 to C32, past saturation, the added latency appears as per-step time while the share before the first token falls, and a capacity argument bounds how far this extends (Section 4).
- *The token budget as a median-versus-tail trade:* at C32 the budget sets the scale of the inter-token latency tail, and the smallest budget moves the delay into the queue (Section 5).

- *What sets the cost of a step:* context, measured directly for single chunks and consistent with a FLOP count, and what the step log could and could not show about scheduled tokens (Section 6).
- *A prediction ledger:* all 59 scored predictions and what the 19 that did not fully hold taught us (Section 8).

The code, configurations, protocols, and reports are public.[1]

2 Background

A request's life in vLLM has three phases: it waits in a *queue*, its prompt is processed (*prefill*), and its output is generated one token per engine step (*decode*). Iteration-level scheduling admits and retires requests between steps [17], and paged KV-cache management keeps memory for many concurrent requests [9]. With *chunked prefill*, a long prompt is split across several steps and shares each step with the running decodes [4]. The *token budget* (`max_num_batched_tokens`) caps the tokens one step may process, decodes plus prompt chunks. vLLM's documentation describes the trade: smaller budgets give better inter-token latency, and larger ones better time to first token and throughput [16]. Our results refine this: at high concurrency, a larger budget improved median inter-token latency and worsened mean time to first token (Section 5).

We use the client-side metrics of vLLM's benchmark. TTFT is the time from sending a request to receiving its first token. Inter-token latency (ITL) is the gap between consecutive streamed tokens of one request. E2E is the total request latency. The *share of latency before the first token* is $\sum \mathrm{TTFT} / \sum \mathrm{E2E}$ over a run's requests. We report ITL percentiles because averages hide jitter [2]: a median and a 99th percentile can rank configurations differently, and tails matter at scale [6]. The benchmark pools the gaps of all requests in a run, and one engine step produces a gap for every request it carries, so a run's ITL p99 reflects its slowest few steps.

3 Method

Setup. All experiments used a rented NVIDIA RTX A4000 (16 GB) with the NVIDIA 580-series driver and vLLM 0.29.0, serving Qwen2.5-0.5B-Instruct. Sessions ran on several rented virtual machines, each with its own physical GPU; Phase C reused Phase B's machine. The client was `vllm bench serve` on the same host, issuing a closed-loop workload. Up to a fixed number of requests (the concurrency, written C1 to C32) are kept in flight, and a new request is sent as soon as one finishes. Each run sends 100 requests (50 for the decode-heavy workload), so a run at C32 starts with a burst of 32 requests and ends as the last ones drain. Prompts are synthetic, with exact input and output lengths and a fixed seed.

Controls. A first baseline (Milestone 1) compared C1 with C8, in forward and reverse order with three repetitions and 15-second cooldowns, and found 6.65× higher output throughput at C8. It ran with prefix caching on, an unpinned model revision, and the model's own sampling defaults (temperature 0.7, top-p 0.8, top-k 20, repetition penalty 1.1), any of which could inflate or mask the effect. A controlled repeat (Milestone 2) turned prefix caching off, pinned the server's model and tokenizer revisions, used greedy sampling, ignored the model's generation defaults (`-generation-config vllm`), and warmed up at each measured concurrency. The effect held (about 6.5×), and these controls are fixed in every later run.

Pre-registration. Each phase's protocol was merged into the public repository before its data were collected. Milestones 1 and 2 stated expected signatures; from Milestone 3 on, each protocol has predictions with thresholds and falsification rules (Phase B states them as signatures for each workload regime). The merge time is the evidence of pre-registration. Each Milestone 3 and 5 report scores every prediction as confirmed, mixed, or not confirmed (with a few qualified verdicts, such as "not testable"), and marks interpretations made after seeing the data as post hoc. Section 8 collects the 59 scored predictions of Milestones 3 and 5.

[1] `https://github.com/pniyongabo/BottleneckShift`

Three views of every run. From Milestone 3 on, each run is measured three ways.

- *Client:* time to first token (TTFT), the gaps between streamed tokens (inter-token latency, ITL; we report the median, p50, and the 99th percentile, p99), end-to-end latency (E2E), and output throughput.
- *Server:* vLLM's Prometheus histograms for queue, prefill, and decode time per request, scraped before and after each run.
- *Resources:* GPU utilization from `nvidia-smi` every 200 ms, and CPU utilization every second.

A per-step scheduler log. vLLM's per-iteration token histogram credits a request's whole prompt to the step that emits its first token, so it cannot show how many prompt tokens a step actually processed. A manipulation check that relied on it failed as specified (Section 5). For Milestone 5 we added an opt-in vLLM plugin that wraps the scheduler and writes one line per engine step, recording the scheduled tokens (split into prefill and decode) and the scheduled requests. Part 2 added each prefill chunk's context position and the time the step's results returned. From these we compute a step's *execution time*: the time from when it could start (it was scheduled and the previous step's results had arrived) until its own results arrived. Unlike the interval between scheduling calls, this measure is tied to the step itself even when vLLM schedules the next step early. vLLM 0.29.0 can schedule a step before the previous step's results return; we did not record which scheduling mode was active.

Validation and provenance. Every series is validated before analysis. The validator checks:

- that runs completed with zero failures and exact token lengths;
- that the server's request count matches the prompts sent;
- that warm-ups ran and telemetry covers the series (sampling gaps of up to 1 s for the GPU and 3 s for the CPU are allowed);
- for Milestone 5, that the step log covers every measured run.

Raw results are archived with a SHA-256 checksum, verified after transfer, before the machine is deleted. Across all sessions the study comprises 18,900 measured requests.

4 Where latency goes under load

Decode dominates each request, even with long prompts. We first compared a prefill-heavy workload (2048 input and 32 output tokens) with a decode-heavy one (64 and 512), at C1 and C8. We had predicted that the prefill-heavy workload would be dominated by the time before the first token. It was not. A request running alone spent about 40 ms in prefill and about 122 ms in its 31 generation steps. A simple cost estimate predicts this [14]. Prefill of 2048 tokens needs about 1.6 TFLOP (two FLOPs per weight per token, plus attention), which takes 21 ms at the A4000's dense 16-bit tensor peak of 77 TFLOPS. Each generation step must read about 1 GB of weights, which takes at least 2.2 ms at 448 GB/s, or 68 ms for 31 steps [10]. At those ideal rates prefill would be about a quarter of the request, so on this GPU the workload we called prefill-heavy is decode-dominated, and the prediction could have been made from the estimate.

At C8 the share of latency before the first token rose from 31% to 45%, but it stayed below half. (At C1 the client's TTFT of 54 ms includes about 14 ms of request handling outside the server's prefill time.) Concurrency did amplify different components in the two regimes. The decode-heavy workload gained 6.46× throughput from C1 to C8 for 10% slower per-token time. The prefill-heavy one gained only 2.8×, with queueing and a tenfold rise in ITL p99.

A plausible reading at the time is that at C8 much of a request's decode time is spent in engine steps that also carry other requests' prompt chunks. Under chunked prefill, vLLM schedules running decodes and new prompt chunks in the same step, up to its token budget. A step that carries a large chunk takes about as long as a prefill, so every request decoding in that step sees a long inter-token gap. A per-request phase breakdown therefore books much of the GPU's prefill work as decode time. Milestone 5's step log fits this reading: at this prompt length, 23% of C8 steps carried prompt work.

Past saturation, the added latency moves into each step. We then raised concurrency on the prefill-heavy workload to C16 and C32, predicting that the time before the first token would come to dominate. The opposite happened. The share of latency before the first token fell from 0.44 at C8 to 0.35 at C16

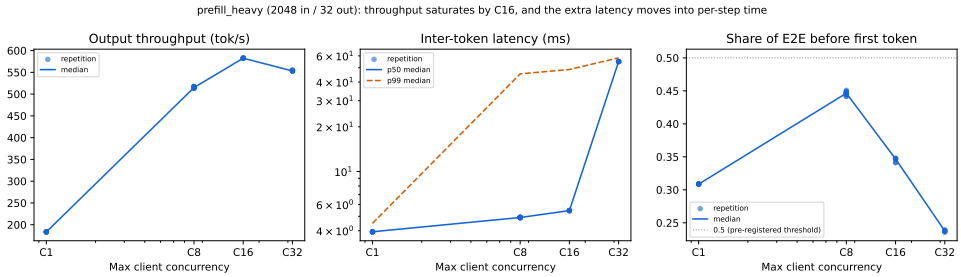


Figure 1: Prefill-heavy workload (2048/32 tokens) against concurrency: output throughput, ITL p50 and p99, and the share of latency before the first token. Past C16, added latency appears as per-step time while the share before the first token falls. The C1 points and six of the twelve C8 points come from the contention study's control series.

and 0.24 at C32. Output throughput peaked at C16 (583 tokens/s) and fell slightly at C32 (555). Median ITL went from 4.9 ms at C8 and 5.5 ms at C16 to 55 ms at C32 (Figure 1). A pre-registered sweep (C8, C12, C16, C24, C32) located the jump between C16 (5.5 ms) and C24 (45.8 ms). It reproduced the earlier session within about 1% on a different machine. The jump in the median is abrupt because the gaps form two clusters, short ones from decode-only steps and long ones from steps that carry a prompt chunk, and the median moves from one cluster to the other once most gaps come from prompt-carrying steps. Server decode time per output token rose more smoothly, from 8.7 ms at C8 to 18.1 ms at C16 and 44.7 ms at C32.

The shift is bounded. At B=2048 a step carries at most one 2048-token chunk, so it costs at most about 56 ms (ITL p99 was 58 ms at C32), and a request's 31 generation steps take at most about 1.8 s. Past saturation throughput no longer grows, so by Little's law each added request in flight adds latency, and once steps are full that latency must go into the wait for the first token. Mean TTFT already doubled from C8 (215 ms) to C32 (431 ms). The per-step shift we measured therefore describes C16 to C32 at this budget, while steps fill toward the cap. Beyond C32, and under open-loop load, we expect delay to return to TTFT; we did not test this.

5 The token budget reshapes per-step cost

vLLM caps the tokens one engine step may process with `max_num_batched_tokens`, the token budget B. Its default on this GPU is 2048, which equalled the prompt length in the experiments above. If the jump in per-step time comes from steps that carry prompt chunks, the budget should control it.

Median against tail. At C32 we set the budget to 512, 2048, and 8192 (Table 1). No setting wins on every metric:

- *512:* spreads prompt work into many small steps. ITL is nearly flat (p50 15.8 ms, p99 17.8 ms), but TTFT triples and throughput falls 10%. Queueing takes two thirds of each request's latency (1.2 of 1.8 s).
- *2048 (the default):* most steps carry a full 2048-token chunk, so p50 and p99 are both near 55–59 ms.
- *8192:* batches several prompts into fewer, much larger steps. The median falls to 6.9 ms and throughput rises 18%, but p99 rises to 169 ms.

The budget with the best median has the worst tail. By mean time per output token after the first, (E2E − TTFT)/31, the order is 512 (15 ms), 8192 (26 ms), and 2048 (45 ms). These descriptions of how each budget cuts prompt work into steps were Phase D's post hoc reading; Milestone 5's step log later showed the step sizes directly: at 2048, 77% of steps carried prompt work, with a median of 2048 prompt-carrying tokens (Section 6).

We had predicted that both ITL p50 and p99 would order 512 < 2048 < 8192. The p99 ordering held and the p50 ordering did not. The manipulation check relied on vLLM's iteration-token histogram, which cannot show per-step chunk sizes (Section 3), and it failed as specified. Under the pre-registered rule, that made the test of the mechanism inconclusive. If we accept server-log evidence that the budgets were applied, which is a deviation from the protocol, the pre-registered form of the step-size explanation, a p50 that rises with the budget, is unsupported. The measured trade-off stands, and the open mechanism motivated Milestone 5.

Table 1: Token budget at C32, prefill-heavy workload (2048/32 tokens). Medians of three repetitions.

Budget	ITL p50 (ms)	ITL p99 (ms)	Mean TTFT (ms)	Output tok/s
512	15.8	17.8	1378.8	496.2
2048	55.4	58.6	432.7	550.8
8192	6.9	168.7	686.1	652.5

Table 2: Prompt length L against budget B at C32 and C8 (32 output tokens). ITL p50 / p99 in ms; medians of three repetitions.

B	L	C32	C8
2048	1024	6.1 / 47.4	4.6 / 40.4
2048	2048	55.3 / 58.3	4.9 / 45.5
2048	4096	57.4 / 64.7	5.7 / 53.8
8192	1024	5.7 / 151.3	4.6 / 60.4
8192	2048	7.0 / 169.5	4.9 / 120.3
8192	4096	9.7 / 199.9	5.5 / 177.5

Prompt length against budget. Milestone 5 varied the prompt length $L \in \{1024, 2048, 4096\}$ at budgets 2048 and 8192 with the step log enabled (Table 2, Figure 2).

- *The budget sets the scale of the tail.* At B=2048, ITL p99 was 58 ms for 2048-token prompts and 65 ms for 4096-token prompts. Past the budget, prompts are split into budget-sized chunks, so the tail grew only 11%. At B=8192 the tail was about 3× higher (170 and 200 ms), while the median stayed below 10 ms.
- *Prompts shorter than the budget did not shrink the tail as predicted.* The pre-registered prediction was that 1024-token prompts would have a tail at least 25% lower than 2048-token prompts. It was 19% lower. The step log suggests why (a post hoc reading): the scheduler packed two 1024-token prompts into one 2048-token step (Section 6).
- *Concurrency moves the median.* At C8 with B=2048, ITL p50 was 4.6–5.7 ms for every prompt length, against 55–57 ms at C32 for $L \geq 2048$ (6.1 ms at L=1024, where prompts are packed), as predicted; only a minority of C8 steps carry prompt work. The tail fell less, to 40–54 ms. 1024-token prompts were packed at C8 as well (an 11% lower tail, against a predicted 25%). At C8 with B=8192, prompt length did move the tail: p99 rose from 60 to 178 ms between L=1024 and L=4096.

6 What sets the cost of a step

Step size. The step log confirmed the budget was applied exactly. No step scheduled more than B tokens, and each run's logged prompt tokens equalled the number sent. The primary prediction of Milestone 5 was that a step's cost tracks its scheduled tokens, measured by the interval from one scheduling call to the next. Pooled over the seven C32 series, steps of at least 1024 tokens were followed by a median interval of 56 ms, against 5.7 ms for steps of at most 64 tokens (9.8×, Spearman 0.87), and the prediction passed. At C8 the same test gave 9.0× but a Spearman correlation of 0.57, below its threshold of 0.7. Under the protocol's rule, this rejects the hypothesis's first clause at C8.

A lagged measure. The interval is confounded with the previous step. vLLM 0.29.0 can schedule step n+1 before it processes step n's results, so the interval credited to step n partly reflects the cost of step $n-1$. The pilot found that the interval correlated better with the previous step's tokens (0.92) than with the step's own (0.87). Some intervals are too short to be a step's own cost: at C8 with B=8192 and L=1024, steps of at least 1024 tokens had a median interval of 16.5 ms, while a single 1024-token chunk took a median of at least 19.5 ms at every context we measured (below). At C32, prompt-carrying steps come in runs, so a lagged measure still separates large steps from small ones, and the C32 pass is weak evidence about a step's own cost. Part 2's execution time is tied to the step itself (Section 3). Over the C8 block it tracked a step's own tokens better than the interval did (Spearman 0.63 against 0.57), still below the threshold. No C32 run

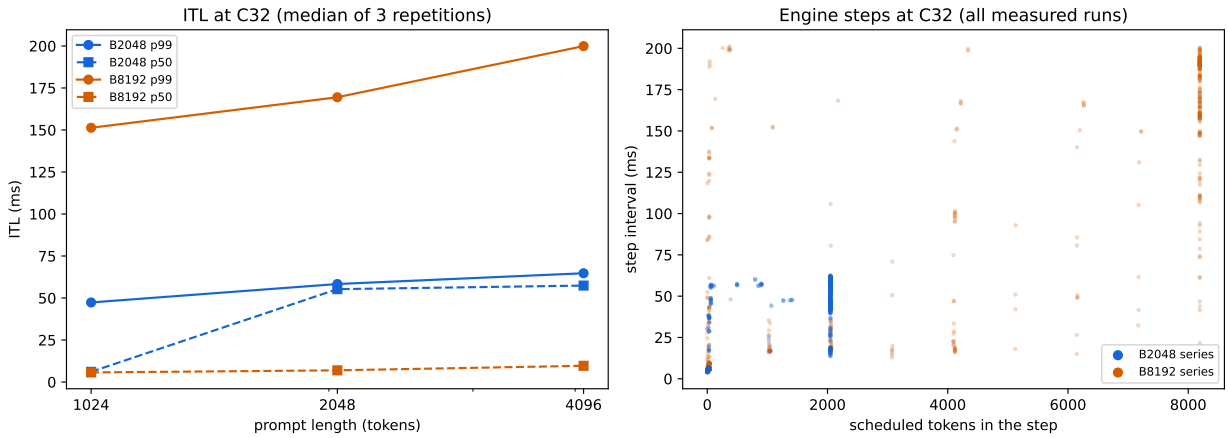


Figure 2: Milestone 5 at C32. Left: ITL p50 and p99 by prompt length for each budget. Right: every measured engine step's interval (to the next step's start) against its scheduled tokens. This interval is confounded with the previous step (Section 6).

recorded it.

Packing (post hoc). The step log also suggests why part of the tail prediction failed. At B=2048 and L=1024, 94% of steps carrying at least 1024 prompt tokens held two or more prompts, and the median prompt-carrying step was a full 2048 tokens. vLLM's scheduler fills the budget whenever prompts are waiting, so a prompt shorter than the budget changes how often steps carry prompt work (38% of steps at L=1024, against 77% at L=2048) more than how large those steps are. This fits the median falling far (6.1 against 55 ms) while the tail fell only 19%. Our workload favors packing: all requests have the same lengths and start together, so they finish and return in groups. With staggered arrivals, fewer prompts would wait at once.

Context. The pilot had suggested a role for context: 2048-token steps were followed by shorter intervals at shorter prompts (45 ms at L=1024, 55.5 ms at L=2048). We withdraw that reading. At both lengths those steps start at context 0, and the gap follows the share of steps that carry prompt work (38% against 77%), as a lagged measure would. A second pre-registered design tested context directly. We ran one request at a time (C1) with 8192-token prompts at budgets 1024 and 2048, so every prompt step held exactly one chunk of exactly B tokens at a known context position, and timed each chunk by its execution time (Figure 3). Chunk cost rose almost linearly with context:

- 1024-token chunks took 25.0 ms at context 0 and 19.5 ms at context 1024, then rose to 35.9 ms at context 7168;
- 2048-token chunks rose from 41.7 ms at context 0 to 67.4 ms at context 6144.

The deepest chunk cost 1.44× and 1.62× the first, and the Spearman correlation between context and execution time was 0.96 in both series. From context 2048 on, each additional 1024 tokens of context added about 1.9 ms to a 1024-token chunk and 3.8 ms to a 2048-token chunk.

These slopes match a FLOP count. A chunk of n tokens at context c spends about $4ncd$ attention FLOPs per layer on the prior context, with d=896 and 24 layers. Each 1024 tokens of context then costs 0.09 TFLOP for a 1024-token chunk and 0.18 TFLOP for a 2048-token chunk. At the tensor peak these take 1.2 and 2.4 ms; at the 40 TFLOPS that the context-0 2048-token chunk achieved, 2.3 and 4.6 ms. The measured slopes lie between the two and double with chunk size, as the count predicts. The first chunk of each prompt was slower than the second at B=1024 (25.0 against 19.5 ms), plausibly because of per-request setup on a request's first step; this reading is post hoc. That setup cost lowers the ratios measured from context 0: taken from the second chunk, the B=1024 ratio is 1.84×, and at context 0 a 2048-token chunk cost 1.67× a 1024-token chunk, where FLOPs predict about 2.1×.

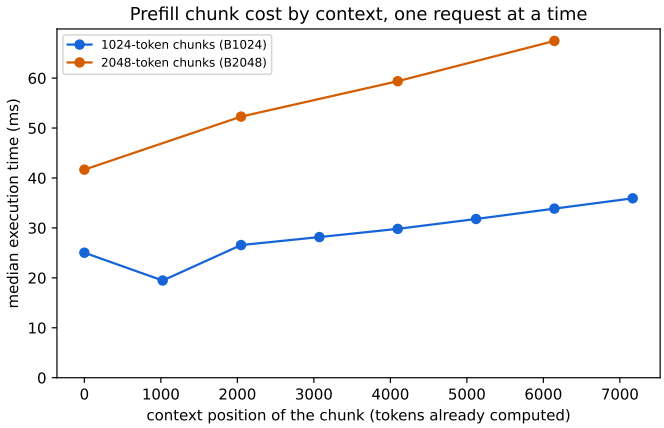


Figure 3: Median execution time of prefill chunks by context position, one request at a time with 8192-token prompts (300 chunks per point).

Table 3: Scored predictions by phase. Phase B scores ten signatures for each of two workload regimes. Milestone 5's C8 predictions are counted in Part 2. Where a report gives a split or qualified verdict (Phase D predictions 2 and 7), we count it as mixed.

Phase	Scored	Confirmed	Mixed	Not confirmed	Not testable
B: workload regimes	20	14	0	5	1
C: concurrency crossover	5	3	0	2	0
C: GPU contention	7	4	2	1	0
D: sweep and token budget	11	6	2	3	0
Milestone 5, C32 block	7	6	0	1	0
Milestone 5, Part 2	9	7	0	2	0
Total	59	40	4	14	1

7 A secondary finding: GPU contention

We also ran a co-located synthetic GPU job alongside the prefill-heavy workload, with a sham condition and a recovery baseline. The job was busy for 25% or 50% of each 100 ms period, and its kernels ran 14% and 28% of the time. The two processes share the GPU by time-slicing [12]. At the 50% setting:

- the job cost 13–15% in end-to-end latency;
- it slowed requests even at C1, and at C1 it slowed prefill more than decode (+22.5% against +11.3%); we have no tested explanation for the difference;
- mean `nvidia-smi` utilization moved by only 1–6 points, against a 3.5-point spread between identical control runs. Without contention it was already about 90% at C1 and higher at C8, which leaves little room to move.

Of seven predictions, four held, two were mixed, and one failed. Utilization stayed within 5 points at C1 but moved 6.0 points at C8. Queue time at C8 rose only 6%, less than prefill (+17%), against the prediction. The share of latency before the first token did not rise, which was the one outright failure. We state this contrast with added load for our closed-loop client; an open-loop test would check it. Near its ceiling, utilization is a poor detector of this interference [11].

8 The prediction ledger

Table 3 counts every scored prediction by phase. Of 59 verdicts, 40 were confirmed, 4 were mixed, 14 were not confirmed, and 1 could not be tested with our instruments. We group the misses below by what caused them; the reports discuss each one, including those that fit no group, such as the order of TTFT across budgets.

The hypothesis was wrong. We predicted that the prefill-heavy workload would be dominated by prefill, which a cost estimate refutes (Section 4), and that time before the first token would dominate past saturation, when the added work went into fuller steps instead. The step-size account predicted that prompts shorter than the budget would make steps smaller; the scheduler packed them, at both C32 and C8. The budget cap itself behaved as predicted.

The instrument could not see the effect. vLLM's iteration-token histogram could not show per-step chunk sizes, so Phase D's manipulation check failed as specified and its step-size test was inconclusive. That miss led to the per-step log. `nvidia-smi` utilization is a busy fraction averaged over the driver's own sample period [11], so no polling rate could resolve 40 ms prefills; we did not replace it. The step log's interval then turned out to be confounded with the previous step (Section 6), and Part 2's execution time corrects this at C8 and C1 only.

The scheduler and the load generator shaped the result. Two queue predictions missed: queue growth past saturation was sub-linear (not confirmed), and contention raised queue time less than prefill time (mixed). Both fit vLLM's policy of scheduling running requests first and filling the rest of the budget with waiting prompts, which turns waiting into longer steps. The closed-loop client also caps how many requests can wait.

A test was too strict for its data. The rank correlation between step size and step interval cleared its threshold at C32 (0.87) but not at C8 (0.57). Forward and reverse condition orders differed by 0.1–2.5% in latency and throughput, enough to fail a strict range-overlap test in the first workload phase. With three runs per order, two ranges fail to overlap by chance one time in ten for each metric even without an order effect, so across about ten metrics a few failures were expected.

Scoring every prediction changed the study. The per-step crossover and the context measurements were tested directly only because an earlier prediction failed or prompted a post hoc reading.

9 Cost

Through October 28, 2026, the study's GPU rental cost $2.04 in total, about one hundredth of a cent per measured request. This covers all seven measurement sessions, their setup, and three machines that failed before collecting data. The cost stayed low because the study used one small model on one mid-range GPU and rented it only for data collection; analysis and writing ran on a laptop.

10 Threats to validity

- *One setup.* All results come from one small model on one GPU type with one vLLM version. With a 0.5B model, a generation step at C1 took about 3.9 ms against a 2.2 ms floor for reading the weights, so host-side and launch work is a large share of each step. Larger models, other GPUs, and other serving systems could change the results qualitatively as well as in scale.
- *Closed-loop load and synthetic prompts.* The client keeps up to a fixed number of requests in flight, which bounds queues. Each run sends 100 requests, so at C32 a run lasts a few seconds, starts with a burst, and ends as the last requests drain; part of each run is below the stated concurrency. All requests have the same lengths and start together, so they move in groups, which may favor packing. Prompts are random tokens, and repetitions replay the same seed, so they measure run-to-run variation only.
- *Few repetitions.* Results are medians of three repetitions, without confidence intervals; the reports give the ranges. Pooled step statistics treat serially correlated steps from three runs as independent.
- *Timing at the scheduler.* Step intervals and execution times are taken at vLLM's scheduler and include host-side work, and the interval is confounded with the previous step (Section 6). We did not check either against GPU-side timing, record vLLM's scheduling mode, or lock GPU clocks.
- *Post hoc readings.* Several explanations are labeled post hoc in the reports, among them packing and the slower first chunk; these remain untested.

- *Several machines.* Sessions ran on several rented machines. Anchor series reproduced earlier results within about 1%, which bounds, but does not remove, machine-to-machine variation.

11 Related work

Orca introduced iteration-level scheduling [17], and vLLM added paged KV-cache management [9]. Sarathi and Sarathi-Serve introduced chunked prefill and the per-step token budget, showed that long prefills stall co-scheduled decodes, and noted that each chunk must read the KV cache of the chunks before it [1, 4]. Our Phase D and Milestone 5 results measure the budget's trade and the per-chunk cost of context directly in vLLM. Vidur predicts iteration time from profiled operator costs, modeling prefill attention by prompt length and decode attention by KV-cache reads [3]; our per-chunk measurements are the kind of data such a model needs for chunked prefill. DistServe and Splitwise run prefill and decode on separate GPUs to avoid this interference [13, 19]; we study it within one engine. Etalon argues that averaged metrics such as time per output token hide jitter [2]. Our median-versus-tail results are an instance, and the tail at scale motivates reporting p99 at all [6]. Hoefler and Belli set out rules for interpretable performance measurement [8]. We follow them in repeating runs, alternating run orders where the design allowed, and reporting every result, but we report no confidence intervals, which three repetitions cannot support. Schroeder et al. show how closed-loop load generators hide queueing [15], which matches our contention and saturation results. Interference from co-located GPU work is well documented [18], as is the weakness of `nvidia-smi` utilization as a measure of GPU load [5, 7, 11]; our contention results are one more instance.

12 Conclusion and future work

On this setup, from C16 to C32 at the default budget, the added delay under load went into per-step time while steps filled toward the budget's cap. With the smallest budget it went into the queue, and past the cap it must. A prompt chunk's cost grows with the context it attends over, in line with a count of attention FLOPs. Whether a step's cost tracks its scheduled tokens under load remains open: the pre-registered test was rejected at C8, and its pass at C32 used a lagged measure. The token budget decides how prompt work is cut into steps, and so trades median against tail latency. Concurrency decides how many steps carry prompt work.

Four directions follow:

- *A step-cost model:* cost as a function of scheduled tokens, context, and KV-cache reads, in the style of Vidur [3], checked against execution time at C32 and tested by whether it predicts ITL p50 and p99 across concurrency and budgets.
- *Open-loop load with varied lengths:* to see whether the per-step account and packing hold when queues can grow and requests do not arrive in groups.
- *Larger models and multi-GPU serving,* where communication adds a third component to step cost.
- *Measurement-driven backend selection,* which we deferred: our single-server data cannot evaluate a selection policy without invented rewards, so it needs a new two-backend experiment.

References

[1] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills, 2023. URL `https://arxiv.org/abs/2308.16369`.

[2] Amey Agrawal, Anmol Agarwal, Nitin Kedia, Jayashree Mohan, Souvik Kundu, Nipun Kwatra, Ramachandran Ramjee, and Alexey Tumanov. Etalon: Holistic performance evaluation framework for LLM inference systems, 2024. URL `https://arxiv.org/abs/2407.07000`.

[3] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for LLM inference. In *Proceedings of Machine Learning and Systems (ML-*

Sys), volume 6, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html.

[4] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134. USENIX Association, 2024. ISBN 978-1-939133-40-3. URL https://www.usenix.org/conference/osdi24/presentation/agrawal.

[5] Roanak Baviskar. GPU utilization is a misleading metric. Trainy Blog, February 2025. URL https://trainy.ai/blog/gpu-utilization-misleading-metric. Published 2025-02-24. Accessed 2026-10-28.

[6] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013. doi: 10.1145/2408776.2408794.

[7] Charles Frye. 'I paid for the whole GPU, I am going to use the whole GPU': A high-level guide to GPU utilization. Modal Blog, February 2025. URL https://modal.com/blog/gpu-utilization-guide. Published 2025-02-24. Accessed 2026-10-28.

[8] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*, pages 73:1–73:12. ACM, 2015. doi: 10.1145/2807591.2807644.

[9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165.

[10] NVIDIA. NVIDIA RTX A4000 datasheet. NVIDIA, . URL https://www.nvidia.com/content/dam/en-zz/Solutions/products/workstations/nvidia-rtx-a4000-datasheet.pdf. Accessed 2026-10-28.

[11] NVIDIA. nvidia-smi. NVIDIA documentation, . URL https://docs.nvidia.com/deploy/nvidia-smi/index.html. Accessed 2026-10-28.

[12] NVIDIA. Time-slicing GPUs in Kubernetes. NVIDIA GPU Operator documentation, . URL https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-sharing.html. Accessed 2026-10-28.

[13] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024. doi: 10.1109/ISCA59077.2024.00019.

[14] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, pages 606–624, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html.

[15] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. Open versus closed: A cautionary tale. In *3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 06)*, pages 239–252. USENIX Association, 2006. URL https://www.usenix.org/legacy/event/nsdi06/tech/schroeder.html.

[16] vLLM Project. Optimization and tuning: Chunked prefill. vLLM documentation. URL https://docs.vllm.ai/en/latest/configuration/optimization.html. Accessed 2026-10-28.

[17] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538. USENIX Association, 2022. ISBN 978-1-939133-28-1. URL `https://www.usenix.org/conference/osdi22/presentation/yu`.

[18] Wei Zhao, Anand Jayarajan, and Gennady Pekhimenko. Tally: Non-intrusive performance isolation for concurrent deep learning workloads. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*, pages 1052–1068. ACM, 2025. doi: 10.1145/3669940.3707282.

[19] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210. USENIX Association, 2024. ISBN 978-1-939133-40-3. URL `https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin`.